

渲染同步比较

1. 几种同步对象

	OpenGL	Metal	Vulkan	
Fence	glFenceSync glWaitSync glClientWaitSync	MTLFence	vk::Fence	Fence主要用于同步对象，管理资源分配和指令同步也包含内存同步
Barrier	glMemoryBarrier barrier memoryBarrier	MTLFence	vkCmdPipelineBarrier	GPU内部同步，针对指令排序，用于同步
Semaphore			vk::Semaphore	GPU内部同步，Queue间同步（即包含内存同步）
Event		MTLEvent MTLSharedEvent	vk::Event	GPU内部同步，介绍

同步类型：

- 1. CPU与GPU之间的同步，一般通过Fence实现
- 2. GPU内部的同步
 - a. 指令同步-保证执行依赖
 - b. 内存同步-保证内存一致性

1.1. OpenGL的同步

a. OpenGL的执行流

i. 异步执行

- 1. 程序提交的OpenGL指令，会先缓存在Host端，后通过一些同步指令批量提交到Device的指令队列中。

ii. 乱序执行

- 1. 现在CPU/GPU为了提高指令执行速度，会将指令在编译器或执行期进行乱序，来提高性能。

iii. 隐式同步

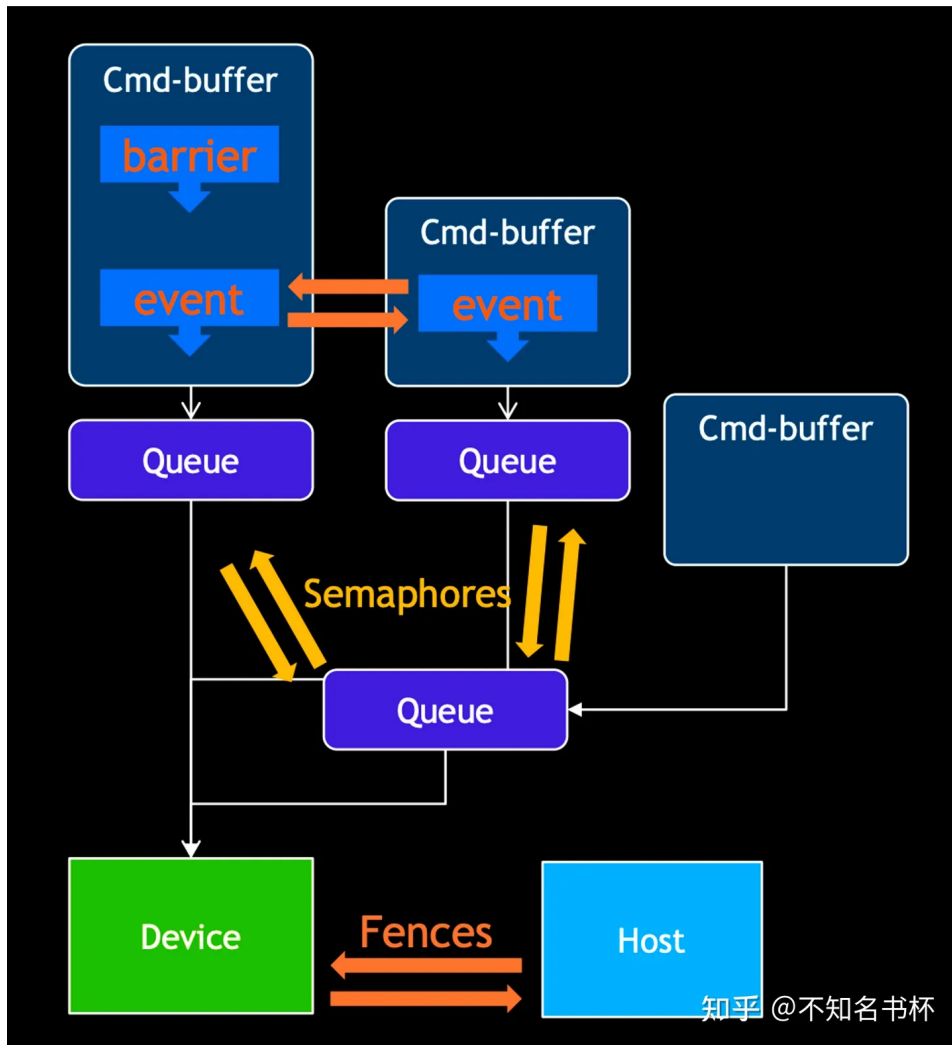
- 1. 在编码中，用户使用OpenGL API的时候，驱动隐式的进行了代码的同步与顺序执行，如：glTexSubImage2D(), glReadPixels(), glBufferSubData()等。

b. 涉及同步的API

```
1  glFlush();
2  glFinsh();
3  //fence
4  glFenceSync();
5  glWaitSync();
6  glClientWaitSync();
7  glDeleteSync();
8  glGetSync();
9  //barrier
10 glMemoryBarrier();
11 //glsl 中
12 barrier();
13 memoryBarrier();
14
15 //解释
16 glFlush():用于把指令发送到device端指令队列。OpenGL中，除同步函数外，函数返回的时候，函数本身并没有执行，只生成了一系列指令，这些指令暂时存放在host端，调用glFlush()函数，程序会阻塞，直至把当前host内的指令都发送到device端指令队列后，解除阻塞，继续执行。
17
18 glFinsh():调用glFinish()函数，程序阻塞，把当前host内的指令都发送到device端指令队列，GPU完成指令队列上所有指令的执行后，解除阻塞，继续执行
19
20 //fence
21 //glFinish()只能保证本线程对应的指令队列中的指令执行完，这就意味着不能在一个线程中等待另一个线程的OpenGL指令执行完。fence可以实现这一点，在一个线程中插入一个fence，然后在另一个线程中就可以去等待这个fence。
22
23 glFenceSync()：往指令队列中插入一个fence。
24 glWaitSync()：调用点后面的代码生成的指令仅存放在host端，不会被发射(issue)到Device指令队列，只有当fence前的指令执行完毕后，才会开始进行指令发射(issue)。
25 glClientWaitSync()：区别于glWaitSync,调用后，线程阻塞，当fence前的指令执
```

1.2. Vulkan的同步

1. 几种同步对象以及同步范围



2. 同步对象解释

a. Fence：同GL一样，主要被用来完成CPU和GPU之间的同步

```
1 // 等待该帧的Fence状态变成signaled
2 vkWaitForFence(currentFrame.fence);
3 vkResetFences(currentFrame.fence);
4
5 // Record Command
6 vkQueueSubmit(work: A, fence: currentFrame.fence);
```

b. Semaphore：主要被用来完成GPU内的同步，可以是Queue内的同步也可以在Queue之间完成同步

```

1 //获取到SwapChain完成此操作后，让SemaphoreA做出Signaled操作
2 vkAcquireNextImageKHR(swapchain,Semaphore:currentFrame.SemaphoreA);
3 //Record Command
4 //...
5 //等待SemaphoreA的完成，提交的所有Command执行完成后让SemaphoreB做出Signaled操作
6 vkQueueSubmit(work: A, waitSemaphore:
  currentFrame.SemaphoreA,singleSemaphore :
  currentFrame.SemaphoreB);
7 //等待SemaphoreB变成Signaled状态之后便可真正展示渲染结果
8 vkQueuePresentKHR(work:A,WaitSemaphores:currentFrame.SemaphoreB)

```

- c. Pipeline Barrier：只能完成Queue内的同步不能够跨Queue，在Pipeline Stage之间树立一个Barrier，要求Barrier前后保持一定的顺序执行。

```

1 //函数定义
2 void vkCmdPipelineBarrier(
3     VkCommandBuffer          commandBuffer,
4     VkPipelineStageFlags     srcStageMask, //指令相关屏障
5     VkPipelineStageFlags     dstStageMask, //指令相关屏障
6     VkDependencyFlags        dependencyFlags,
7     uint32_t                 memoryBarrierCount,
8     const VkMemoryBarrier*    pMemoryBarriers,
9     uint32_t                 bufferMemoryBarrierCount,
10    const VkBufferMemoryBarrier* pBufferMemoryBarriers,
11    uint32_t                 imageMemoryBarrierCount,
12    const VkImageMemoryBarrier* pImageMemoryBarriers);
13
14

```

- d. Event：可用于在提交给同一Queue的Command之间或在CPU和GPU之间插入细粒度的依赖关系

```

1 void vkCmdSetEvent(
2     VkCommandBuffer      commandBuffer,
3     VkEvent               event,
4     VkPipelineStageFlags stageMask);
5
6 void vkCmdWaitEvents(
7     VkCommandBuffer      commandBuffer,
8     uint32_t             eventCount,
9     const VkEvent*       pEvents,
10    VkPipelineStageFlags srcStageMask,
11    VkPipelineStageFlags dstStageMask,
12    uint32_t              memoryBarrierCount,
13    const VkMemoryBarrier* pMemoryBarriers,
14    uint32_t              bufferMemoryBarrierCount,
15    const VkBufferMemoryBarrier* pBufferMemoryBarriers,
16    uint32_t              imageMemoryBarrierCount,
17    const VkImageMemoryBarrier* pImageMemoryBarriers);

```

- e. SubPass Dependency: 隐式同步源语，在RenderPass中的SubPassDependency来体现，与Pipeline Barrier类似，专门用于表达SubPass之间的依赖关系以及RenderPass中的Command和外部Command的依赖关系。

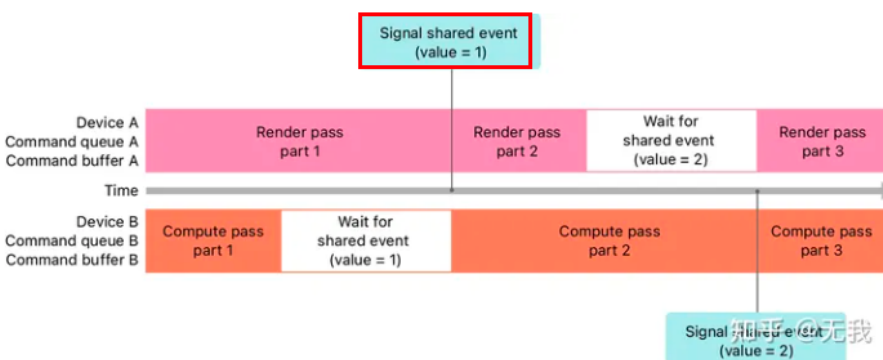
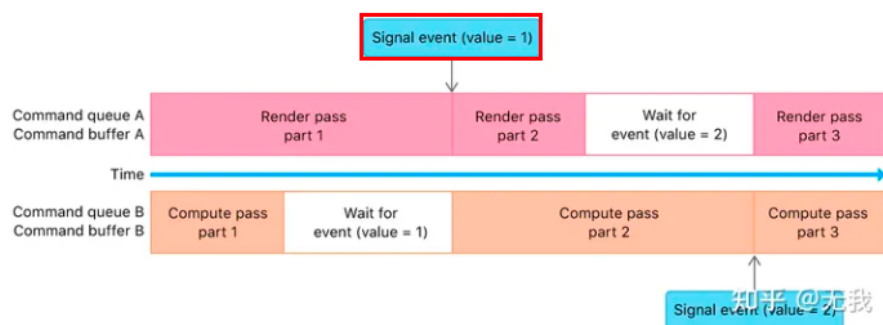
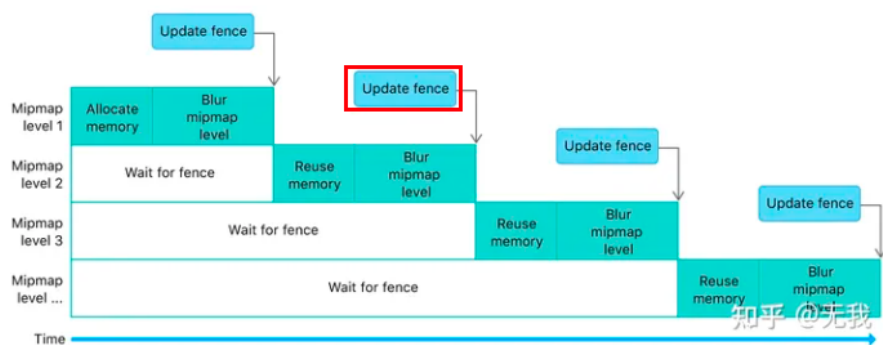
```

1 typedef struct VkSubpassDependency {
2     uint32_t srcSubpass;
3     uint32_t dstSubpass;
4     VkPipelineStageFlags srcStageMask;
5     VkPipelineStageFlags dstStageMask;
6     VkAccessFlags srcAccessMask;
7     VkAccessFlags dstAccessMask;
8     VkDependencyFlags dependencyFlags;
9 } VkSubpassDependency;

```

1.3. Metal的同步

1. 几种同步对象以及范围(MTLFence, MTLEvent, MTLSharedEvent)



2. **MTLFence** : 同一个Queue中的同步, 常用于跟踪和管理跨CommandEncode之间的依赖。当不同的Command会对资源进行读写时, 就需要保证资源之间依赖是正确的, MTLFence可以做到捕获特定阶段之前的 GPU 工作, 当GPU遇到MTLFence时, 它必须等到所有捕获到该阶段的工作完成后才能继续执行。

官方demo :

[Implementing a Multistage Image Filter Using Heaps and Fences | Apple Devel...](#)

3. **MTLEvent**: 用于 GPU 内部的同步,是可以跨Queue同步的。

官方Demo :

[Implementing a Multistage Image Filter Using Heaps and Events | Apple Devel...](#)

```
1  _event = [_device newEvent];
2  _commandQueueA = [_device newCommandQueue];
3  _commandQueueB = [_device newCommandQueue];
4
5  - (void)renderFrame
6  {
7      id<MTLCommandBuffer> commandBufferA = [_commandQueueA
commandBuffer];
8      // 设置Event的值为1
9      [commandBufferA encodeSignalEvent:_event value:1];
10     //...
11     // 等待Event的值大于等于2
12     [commandBufferA encodeWaitForEvent:_event value:2];
13     [commandBufferA commit];
14
15     // Command Queue B (Compute Processing)
16     id<MTLCommandBuffer> commandBufferB = [_commandQueueB
commandBuffer];
17     [commandBufferB encodeWaitForEvent:_event value:1];
18     //...
19     [commandBufferB encodeSignalEvent:_event value:2];
20     [commandBufferB commit];
21 }
```

Ref :

1. [🔗 理解Vulkan同步\(Synchronization\) - 知乎](#)
2. [🔗 浅谈Metal中的同步 - 知乎](#)